

Arhitektura računala

INSTRUKCIJE MIKROUPRAVLJAČA

GRAĐA MIKROUPRAVLJAČA

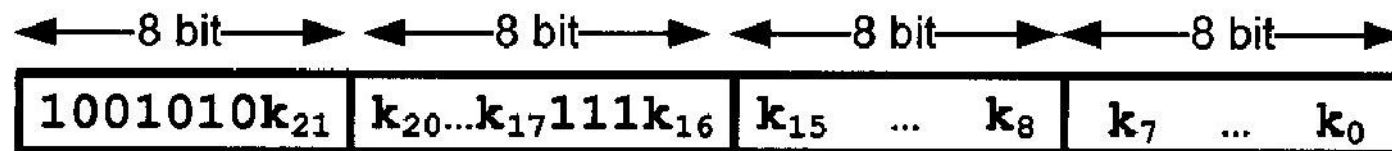
- CALL naredba koja se koristi za pozivanje potprograma.
 - Potprogrami se često koriste za izvršavanje zadataka koje treba ponavljati.
 - To čini program strukturiranijim uz uštedu prostora u memoriji.
 - U AVR-u postoje četiri naredbe za poziv potprograma
 - CALL (dugi poziv)
 - RCALL (relativni poziv),
 - ICALL (neizravno pozivanje Z registra) i
 - EICALL @xtended neizravni poziv Z).
 - Izbor koji ćete koristiti ovisi o ciljanoj adresi.

GRAĐA MIKROUPRAVLJAČA

- CALL 32-bitna instrukcija
 - 10 bita kod operacije + 22 bita k_{21} - k_0 adresa potprograma (kao i kod JMP)
 - Može se adresirati 4M adresni prostor (000000-3FFFFFF)

1001	010 k_{21}	$k_{20}k_{19}k_{18}k_{17}$	111 k_{16}
$k_{15}k_{14}k_{13}k_{12}$	$k_{11}k_{10}k_9k_8$	$k_7k_6k_5k_4$	$k_3k_2k_1k_0$

$$0 \leq k \leq 3FFFFFF$$



$$0 \leq k < 4M$$

GRAĐA MIKROUPRAVLJAČA

- CALL

- Povratak iz pozvanog potprograma → mikroupravljač sprema adresu instrukcije ispod instrukcije CALL
- Pozivom potprograma procesor sprema sadržaj sljedeće instrukcije na stog i započinje dohvaćanje sljedeće instrukcije
- Nakon završetka potprograma, pomoću instrukcije RET kontrola se prenosi natrag na pozivatelja

GRAĐA MIKROUPRAVLJAČA

- Ako je stog dio RAM-a, unutar CPU-a mora postojati registar koji ukazuje na njega. Registar koji se koristi za pristup stogu naziva se registar SP (pokazivač stoga).
- U I / O memorijskom prostoru postoje dva registra koja se nazivaju SPL (niži bajt SP-a) i SPH (viši bajt Sp-a)
- SP je realiziran kao dva registra. SPH registar predstavlja gornji-viši bajt SP-a, dok SPL registar predstavlja donji-niži bajt.



GRAĐA MIKROUPRAVLJAČA

- Pokazivač stoga mora biti dovoljno širok da adresira sav RAM.
- AVR-ovi s više od 256 bajtova memorije SP je napravljen od dva 8-bitna registra (SPL i SPH), dok je u AVR-ima s manje od 256 bajtova SP napravljen samo od SPL-a, jer 8-bitni registar može adresirati 256 bajtova memorije.
- Pohranjivanje informacija poput brojača programa na stogu naziva se PUSH, a učitavanje sadržaja stoga natrag u CPU registar naziva se POP.
- Pokazivač stoga (SP) pokazuje na vrh stoga (TOS). Dok postavljamo podatke na stog, podaci se spremaju tamo gdje pokazuje SP, a SP se umanjuje za jedan.

GRAĐA MIKROUPRAVLJAČA

- Za postavljanje sadržaja registra na stog koristimo instrukciju PUSH..
- PUSH Rr; Rr može biti bilo koji od registara opće namjene (R0-R31)
- Na primjer, za pohranu vrijednosti R10 možemo napisati sljedeću instrukciju:
 - PUSH R10 ; →; pohranite R10 na stog i smanjite sadržaj SP za jedan

GRAĐA MIKROUPRAVLJAČA

- Vraćanje sadržaja stoga u zadani registar suprotan je → POP
- Kada se izvrši POP naredba, SP se povećava i iz gornje adrese stoga sadržaj se kopira natrag u registar. To znači da je stog LIFO (Last First-Out) memorija.
- Da bismo dohvatili bajt podataka iz stoga, koristimo POP instrukciju.
- POP Rr; Rr može biti bilo koji od registara opće namjene (R0 - R31)
- Na primjer, sljedeća instrukcija skače s vrha stoga i sadržaj kopira u R10:
- POP R10; povećanje SP za jedan, a zatim stavlja sadržaj na vrha stoga u R10

GRAĐA MIKROUPRAVLJAČA

Stog i pokazivač stoga te registre koji se koriste nakon izvršavanja svake naredbe.

```
.INCLUDE "M32DEF.INC"
.ORG 0
;initialize the SP to point to the last location of RAM (RAMEND)
LDI R16, HIGH(RAMEND) ;load SPH
OUT SPH, R16
LDI R16, LOW(RAMEND) ;load SPL
OUT SPL, R16
```

```
LDI R31, 0
LDI R20, 0x21
LDI R22, 0x66
```

```
PUSH R20
PUSH R22
```

```
LDI R20, 0
LDI R22, 0
```

```
POP R22
POP R31
```

Solution:

After the execution of	Contents of some of the registers				Stack
	R20	R22	R31	SP	
OUT SPL,R16	\$0	\$0	0	\$085F	
LDI R22, 0x66	\$21	\$66	0	\$085F	
PUSH R20	\$21	\$66	0	\$085E	
PUSH R22	\$21	\$66	0	\$085D	
LDI R22, 0	\$0	\$0	0	\$085D	
POP R22	\$0	\$66	0	\$085E	
POP R31	\$0	\$66	\$21	\$085F	

GRAĐA MIKROUPRAVLJAČA

- U AVR assembleru RAMEND predstavlja adresu zadnje RAM lokacije
- Dakle, ako želimo inicijalizirati SP tako da pokazuje na posljednju memorijsku lokaciju možemo jednostavno učitati RAMEND u SP. Primijetili ste da se SP sastoji od dva registra, SPH i SPL. Dakle, viši bajt RAMEND-a učitavamo u SPH, a niži bajt RAMEND-a u SPL.

GRAĐA MIKROUPRAVLJAČA

PRIMJER

- Kod poziva potprogram, procesor prvo sprema adresu naredbe odmah ispod naredbe CALL na stog, a zatim prenosi kontrolu na taj potprogram. Tako CPU zna gdje nastaviti kad se vrati iz pozvanog potprograma.
- Za AVR-ove čiji brojač programa nije duži od 16 bitova (npr. ATmega128, ATmega32), vrijednost programskog brojača podijeljena je u 2 bajta. Prvo se viši bajt spremi na stog, a zatim niži bajt.

GRAĐA MIKROUPRAVLJAČA

PRIMJER

- Kad se izvrši naredba RET na kraju potprograma, gornja adresa stoga kopira se natrag u programski brojač i povećava se pokazivač stoga za jedno mjesto.
- Kada se izvrši naredba CALL, adresa naredbe ispod naredbe CALL postavlja se na stog; tako, kada se izvršavanje funkcije završi i izvrši RET, adresa naredbe ispod CALL učitava se u PC, i zatim se izvršava instrukcija ispod naredbe CALL.

GRAĐA MIKROUPRAVLJAČA

- stog možemo definirati bilo gdje u memoriji opće namjene.
- u AVR-u stog može biti velik poput njegovog RAM-a
- u AVR-u stog se koristi za pozive i prekide.
 - nakon pozivanja potprograma stog prati gdje bi se CPU trebao vratiti nakon dovršenja potprograma → oprez kada manipuliramo sadržajem stoga.

GRAĐA MIKROUPRAVLJAČA

- Programiranje – jedan glavni program i mnogo potprograma
- Mogućnost testiranja pojedinih dijelova programa (potprograma) – skraćeno vrijeme razvoja
- CALL ciljna adresa programa može biti bilo gdje unutar 4M memorijskog prostora AVR-a

```
.INCLUDE "M32DEF.INC" ;Modify for your chip

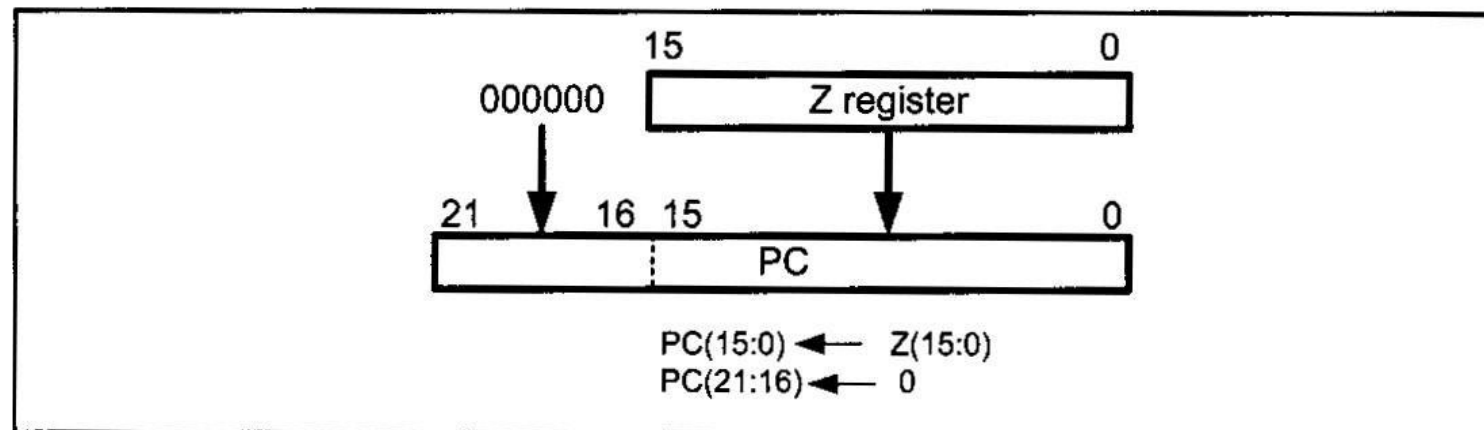
;MAIN program calling subroutines
        .ORG 0
MAIN:    CALL SUBR_1
        CALL SUBR_2
        CALL SUBR_3
        CALL SUBR_4
HERE:    RJMP HERE      ;stay here
;-----end of MAIN
;
SUBR_1:   ....
        ....
        RET
;-----end of subroutine 1
;
SUBR_2:   ....
        ....
        RET
;-----end of subroutine 2
;
SUBR_3:   ....
        ....
        RET
;-----end of subroutine 3
;
SUBR_4:   ....
        ....
        RET
;-----end of subroutine 4
```

RCALL

- dvobajtna – za adresu koristi samo 12 bitova
- adresa potprograma mora biti unutar -2048 do +2047 riječi memorije u odnosu na adresu trenutnog PC
- CALL vs RCALL – može uštedjeti određeni broj bajtova

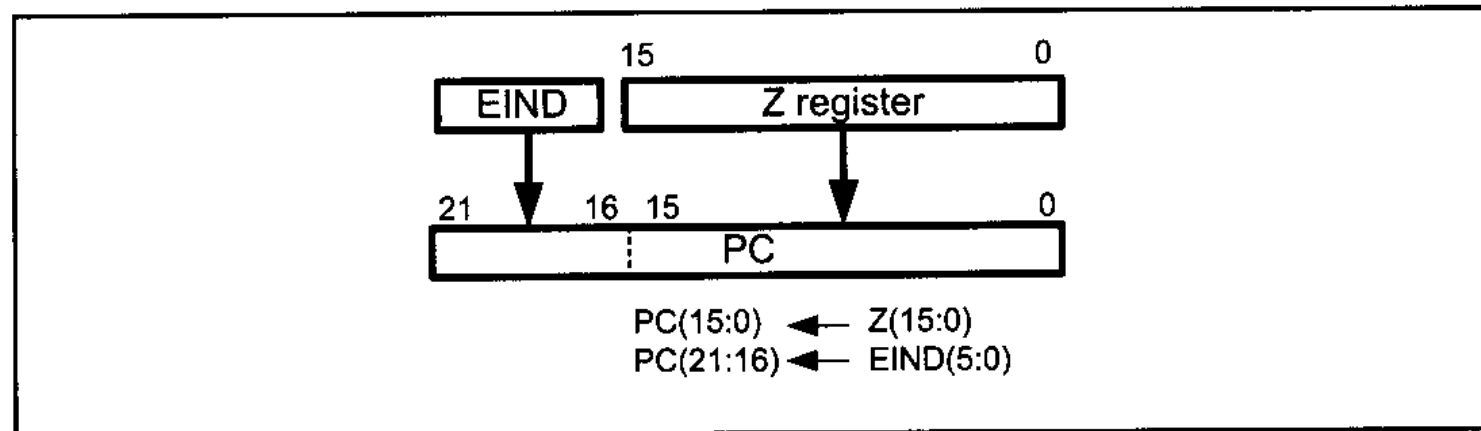
ICALL

- Z registar – specificira ciljnu adresu, sadržava adresu funkcije kada se izcršava naredba ICALL
- ICALL – može pozvati potprograme unutar najnižih 64K riječi programske memorije



EICALL

- EICALL – prošireni neizravni poziv
 - Učitava Z registar u donjih 16 bitova PC + EIND u gornjih 6 bitova PC-a
 - EIND dio memorije I/O

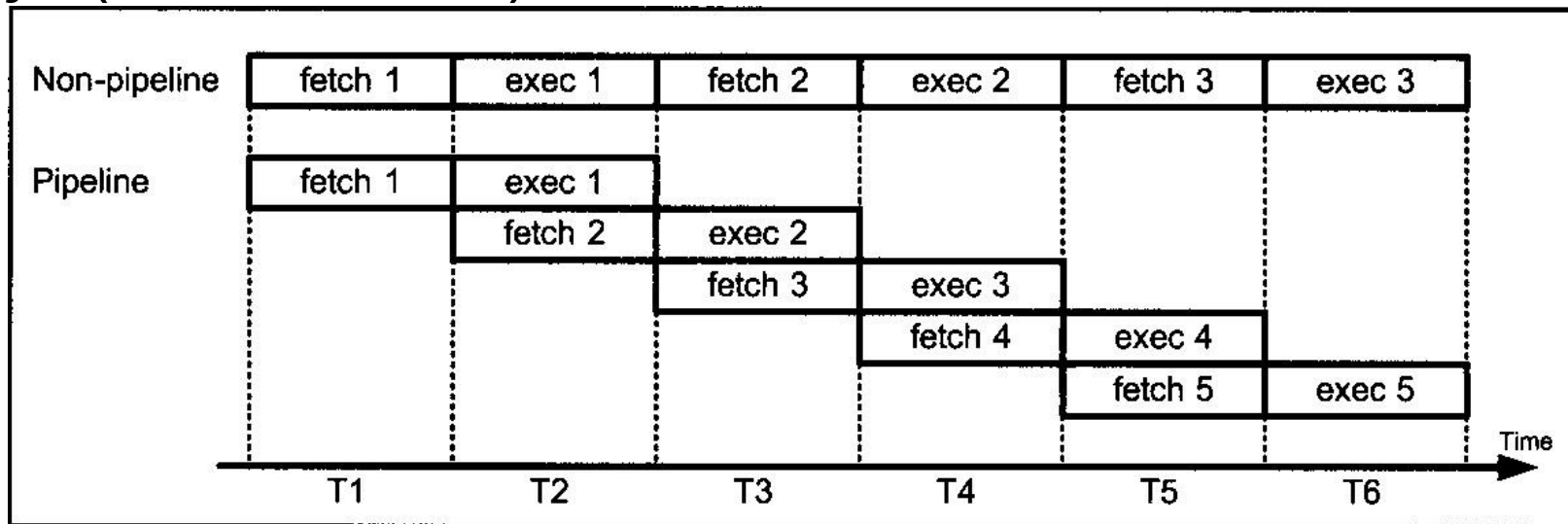


Kašnjenje

- Kreiranje vremenskog kašnjenja → 2 čimbenika koja utječu
 - Frekvencija kristalnog oscilatora spojenog na pinove XTAL1 i XTAL2
 - Dizajn AVR-a

Rapoređivanje zadataka

- 8085 – dohvaća 1 instrukciju → izvršava 1 instrukciju
→ dohvaća 2 instrukciju → izvršava 2 instrukciju...
- AVR dohvaća 1 instrukciju → izvršava 1 i dohvaća 2 instrukciju (istovremeno) → izvršava 2 i dohvaća 3 instrukciju (istovremeno)...



HVALA NA PAŽNJI!